

VLSI semester recap

by Erik Girdwaskus

→ like DDCA last semester, this won't be very structured → I might include parts of the book that the lectures are based on, but as of WL there has been no need for that yet

→ also, the exercises are a massive part of this, and that's not something my theory can cover

→ color code: topics, anything else that's important

Field-Programmable Logic:

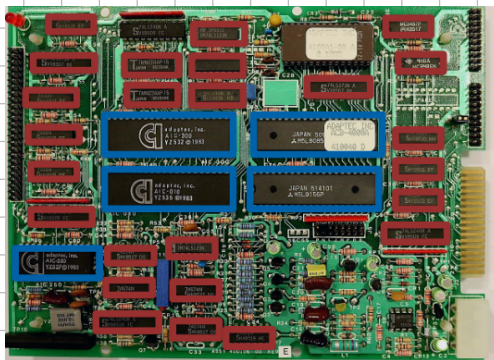
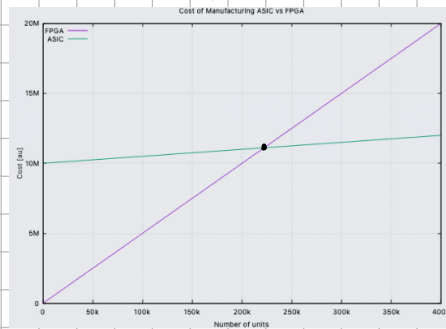
→ ASICs and semi-custom IC's undergo photolithography so that, to some degree, logic/designs are pre-programmed onto semiconductor wafers

→ FPGAs do not use this and are programmed electrically

→ ASICs are designed using HDL and manufactured according to that description; commercial FPGAs can be programmed to fulfill a multitude of logic functions using HDL

	Processor	FPGA	ASIC
Readily available	yes	yes	no
Can be customized	no	yes	yes
Initial cost	minimal	minimal	millions
Unit cost	medium (1-1000)	high (40 - 10'000)	very little (0.5 - 5)
Performance	Decent	Better	Best
Programming	C/C++	HDL VLSI1	HDL
Time to program	seconds	seconds	months

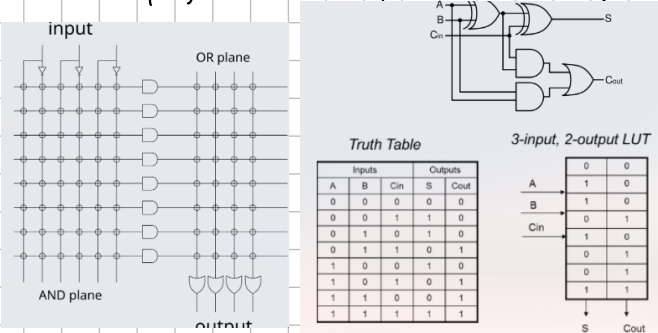
→ for finalized, mass-produced designs, ASIC becomes worth it from a breakeven point



→ before FPGA: ICs provide main functionality, glue logic (AND/OR...) in between took up large portion of PCB

→ in this era, simple gates like inverters or NAND gates were mass-produced and used in glue logic

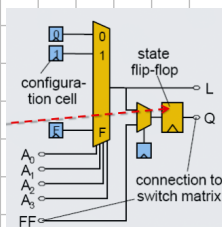
PLA's: programmable to any logic function using minterms/maxterms → early days: one-time programmable using fuses



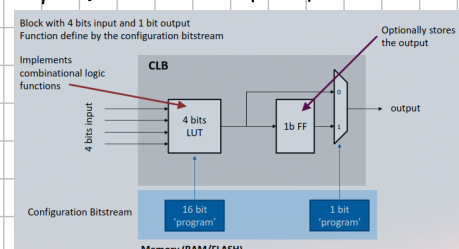
→ comb. logic represented by TT → implement this in a small memory called a lookup table (LUT) → implement a function by writing all possible values of the function in the LUT → addressed with input values

→ Limitation: # inputs

LUT = FF + MUX: if it is a MUX evaluating the TT stored in SRAM cells → for seq. logic, add a flip-flop → circuit can use output from LUT or FF →



Combinational Logic Block (CLB):



basic FPGA: consists of CLBs, wires connecting elements (logic/clock), IO pads (reading signals in/out of FPGA)

connecting logic cells:

- Tri-state buffers (TSB) decide if there is connection from CLB outputs/
- Mux selects which wires serve as CLB inputs → routing adjacent signals
- **switch box:** crossing points between CLB's, not at their boundary

→ switch boxes are **programmable** w.r.t existence and direction of connections

→ they have both short channels connecting adj. CLB's and long channels for further-away CLB's → connection number/mix specified by vendor

→ the flexibility of FPGA requires **extra cost** (power/cost/delay), so we **lose performance** compared to ASIC, but FPGAs are viable as commercial FPGAs are produced in **much smaller** (~10nm) processes than ASICs (~65nm)

→ FPGA has to support multiple I/O voltages, connection speeds, delays, standard interfaces

→ **IO pads** connect PCB signals to CLBs, organized in banks (number of IOB per bank depends

on technology/producers)

→ all pads in one bank **share a supply voltage**

→ there are **special pads** for ground (GND), supplies (VCC, VCCINT, ...), clocks, programming (JTAG)

→ IO blocks support **many standards** (LVTTTL, LVCMOS, LVDS, ...), both single-ended & differential

→ they have sync. and delay adjustment capabilities

→ **wiring size of CLBs keeps increasing** → many designs use same blocks → **integrate them into the FPGA!**

→ FFs in logic blocks can be used as mem. (1, 2 FFs per CLB), modern FPGAs have dedicated SRAM (Xilinx/AMD uses 36kbit BRAMs with 32k default + 4k parity), fully programmable

Our exercise FPGA: Xilinx Zynq Z7 → Artix-7 family with this CLB layout:

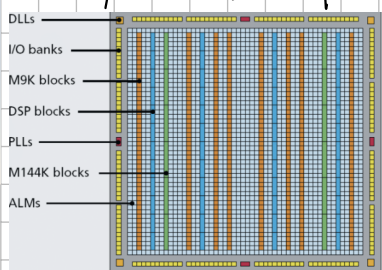
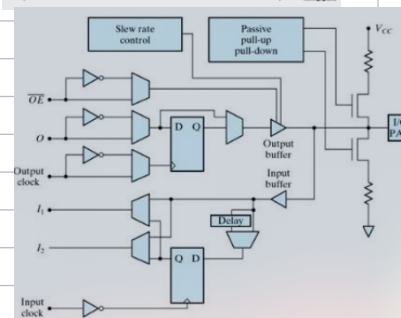
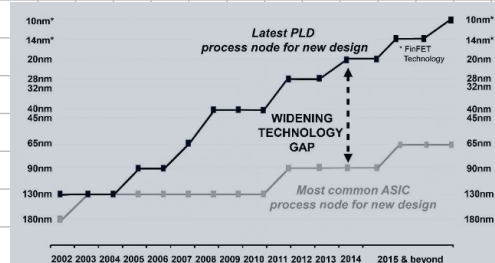
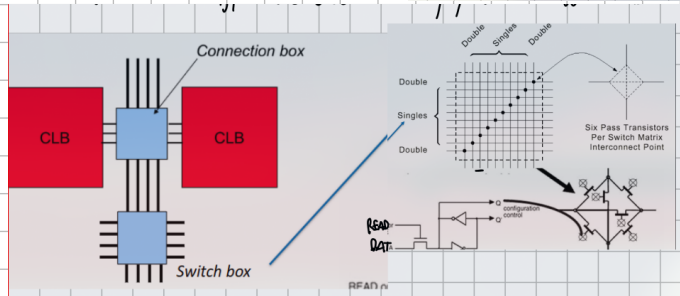
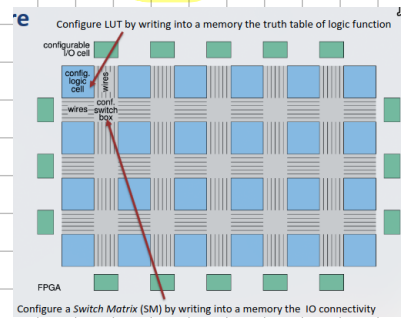
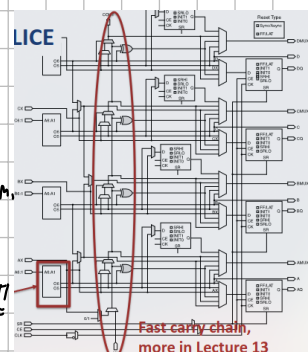
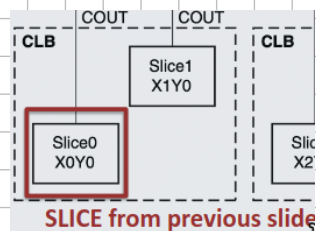
→ LUT/FF combined into **SLICE** to optimize for "the common case"

→ CLBs consist of SLICES: there are 2 types, SLICEL for logic, SLICEM for mem.

FPGA design process:

→ describe desired behavior using HDL ⇒

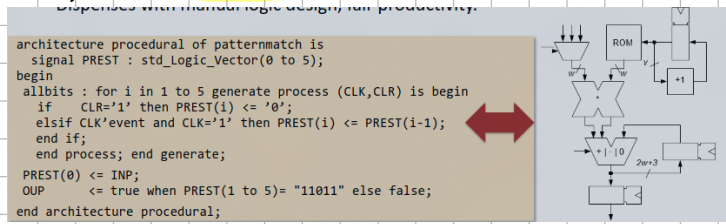
Use design tools to **generate bitstream** (map functions to logic blocks,



connect logic blocks through routing, Config-bits as a whole form bitstream) $\Rightarrow$ after this, the FPGA works like a designed HW

$\rightarrow$ hand-drawing circuits or even schematics is **niche/outdated**, more relevant abstraction step is RTL synthesis of RTL to netlist, above that level of abstraction are Virtual Components (purchased RTL code) and High-Level Synthesis (C or similar SW code $\rightarrow$ netlist)

Register-Transfer-Level Design: define circuit as network of stateful elements (LUT/FF) held together by CL (technology-agnostic)

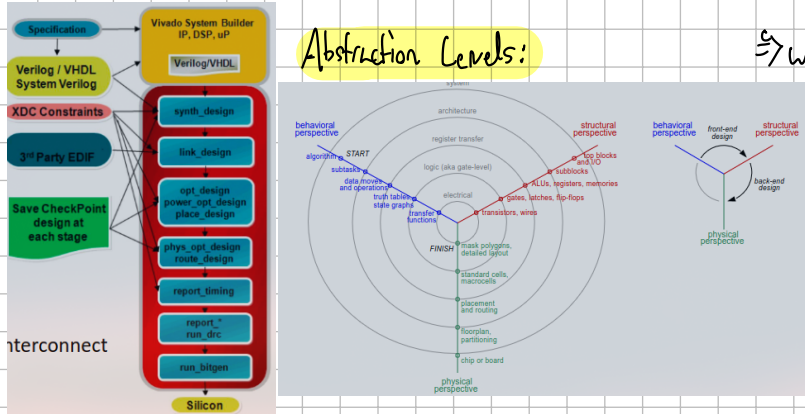


FPGA design workflow: specifications $\Rightarrow$ RTL design $\Rightarrow$ synthesis

(maps RTL code to logic blocks) $\Rightarrow$ optimize design (check timing, resource)

$\Rightarrow$ place and route (which logic blocks are used? configure the interconnect)

$\Rightarrow$ write bitstream to FPGA



Abstraction Levels:

$\rightarrow$ for **automated design** (plenty of room at the top)

System-level design: (most influential) specify function/characteristics of system $\Rightarrow$ partition functionality into subtasks $\Rightarrow$ explore HW/SW tradeoffs $\Rightarrow$ decide on make or buy for all building blocks $\Rightarrow$

decide on interfaces/protocols for data exchange $\Rightarrow$ decide on data types, operating modes, exception handling, ... $\Rightarrow$ define, model, eval., refine subtasks $\Rightarrow$ result: System-level model

Architecture design I: (high-level decisions) Partition comp. task according to HW $\Rightarrow$ coordinate subtasks $\Rightarrow$ allocate HW resources to subtasks $\Rightarrow$ define datapaths/controllers $\Rightarrow$ decide betw. on-chip, off-chip RAM and registers $\Rightarrow$ decide on communication protocols (parallel, serial) $\Rightarrow$ define degree of HW ^(par.) parallelism $\Rightarrow$ define degree of pipelining $\Rightarrow$ decide on circuit style/fabrication process $\Rightarrow$ estimate circuit size/cost $\Rightarrow$ result: HL block diagram, preliminary floor plan

Algorithm design: (comp.) optimize computation for HW: cut down on comp. burden, mem. reqs. $\Rightarrow$ compromise betw. comp. complexity and accuracy $\Rightarrow$ contain effects of finite word length comp. $\Rightarrow$ decide how to represent numbers $\Rightarrow$ eval. alternatives $\Rightarrow$ quantify min. reqs. for comp. resources $\Rightarrow$ bit-true SW model

Architecture design II: (lower-level details) Implementation of ALUs $\Rightarrow$ use hardwired logic or microcode + controller $\Rightarrow$ ROM vs. random logic $\Rightarrow$ what ops. to perform in which clock cycle (scheduling) $\Rightarrow$ what ops. to perform on which processing unit (binding) $\Rightarrow$ what clocking discipline? $\Rightarrow$ what time interval to use as basic clock period? $\Rightarrow$ when to use bidirectional or unidirectional bus? $\Rightarrow$ how to control access to bus with multiple drivers? $\Rightarrow$ how to perform eval./testing? $\Rightarrow$ how to initialize circuit? $\Rightarrow$ result: detailed diagrams & verified RTL code